08. 수학적 기초 (Mathematical Foundations)

8.1 유한체 (Finite Fields)

8.1.1 유한체란?

유한체는 유한개의 원소를 가진 체(Field)입니다.

체의 공리:

1. 덧셈과 곱셈이 정의됨
2. 덧셈과 곱셈에 대한 항등원 존재 (0, 1)
3. 덧셈과 곱셈에 대한 역원 존재
4. 교환법칙, 결합법칙, 분배법칙 성립

표기: $\mathbb{F}_p$ (p개 원소를 가진 유한체)

8.1.2 Mersenne31 필드

정의:

```
p = 2³¹ - 1 = 2,147,483,647

𝔽_p = {0, 1, 2, ..., p-1}
```

연산:

```
덧셈: (a + b) mod p

뺄셈: (a - b) mod p
```

```
곱셈: (a × b) mod p

나눗셈: a × b⁻¹ mod p (b⁻¹은 b의 곱셈 역원)
```

예시:

```
a = 10, b = 20 (in 𝔽_p)

a + b = 30

a × b = 200

a / b = a × b⁻¹ = 10 × (20의 역원)
```

8.1.3 왜 유한체를 사용하는가?

1. **오버플로우 없음**: 모든 연산 결과가 필드 내에 존재
2. **역원 존재**: 0이 아닌 모든 원소는 곱셈 역원 보유
3. **효율적 연산**: 컴퓨터에서 빠르게 계산 가능
4. **암호학적 안전성**: 이산 로그 문제 등의 어려운 문제 기반

8.1.4 확장 필드 (Extension Fields)

정의:

```
𝔽_p³ = 𝔽_p[α] / (α³ - ω)

원소: a₀ + a₁ ·α + a₂ ·α² (a₀, a₁ , a₂  ∈ 𝔽_p)
```

BinomialExtensionField<Mersenne31, 3>:

- Mersenne31의 3차 확장
- 원소 개수: $(2^{31}-1)^3$

- 더 큰 보안 수준 제공

연산 예시:

```
a = a₀ + a₁ ·α + a₂ ·α²

b = b₀ + b₁ ·α + b₂ ·α²

a + b = (a₀+b₀) + (a₁ +b₁ )·α + (a₂ +b₂ )·α²
```

8.2 다항식 (Polynomials)

8.2.1 다항식 기초

정의:

```
p(x) = a₀ + a₁ ·x + a₂ ·x² + ... + aₙ·xⁿ

차수: deg(p) = n (aₙ ≠ 0)
```

평가:

```
p(5) = a₀ + a₁ ·5 + a₂ ·25 + ... + aₙ·5ⁿ
```

8.2.2 다항식 보간 (Polynomial Interpolation)

문제: n+1개의 점 $(x_0, y_0), \ldots, (x_n, y_n)$을 지나는 차수 n 이하의 다항식 찾기

라그랑주 보간:

```
p(x) = Σᵢ yᵢ · Lᵢ(x)
```

```
여기서 Lᵢ(x) = Πⱼ≠ᵢ (x - xⱼ) / (xᵢ - xⱼ)
```

유일성: 차수 n 이하의 다항식은 유일하게 결정됨

zkVM에서의 사용:

```
추적 테이블의 각 컬럼 → 다항식

행 인덱스 → 평가 지점
```

예시:

```
컬럼 0: [0, 10, 20, 30]

↓ 보간

p₀(x): p₀(0)=0, p₀(1)=10, p₀(2)=20, p₀(3)=30

p₀(x) = 10x
```

8.2.3 FFT (Fast Fourier Transform)

역할: 다항식 평가 및 보간을 O(n log n)에 수행

기본 아이디어:

```
p(x) = a₀ + a₁ ·x + a₂ ·x² + ... + aₙ₋₁ ·xⁿ⁻¹

p(ω⁰), p(ω¹), ..., p(ωⁿ⁻¹) 계산 (ω는 n차 단위근)
```

단위근:

```
ωⁿ = 1
```

$\omega, \omega^2, \ldots, \omega^{n-1}, \omega^n=1$

Plonky3에서의 사용:

- 추적을 다항식으로 변환
- 다항식을 확장된 도메인에서 평가
- FRI 프로토콜에서 핵심 역할

8.3 Reed-Solomon 코드

8.3.1 Reed-Solomon 코드란?

정의:

```
메시지: (m₀, m₁ , ..., mₖ₋₁ )

다항식: p(x) = m₀ + m₁ ·x + ... + mₖ₋₁ ·xᵏ⁻¹

코드워드: (p(x₀), p(x₁ ), ..., p(xₙ₋₁ ))
```

특징:

- 차수 k-1 다항식을 n개 점에서 평가
- 최대 n-k개 오류 검출 가능
- 최대 (n-k)/2개 오류 정정 가능

8.3.2 낮은 차수 테스트 (Low-Degree Test)

문제: 주어진 평가 값들이 낮은 차수 다항식에서 나왔는지 확인

방법:

1. 무작위 위치에서 값 쿼리

2. 다항식 보간 시도

3. 보간된 다항식이 쿼리 값과 일치하는지 확인

zkVM에서의 역할:

- 추적이 올바른 다항식으로 인코딩되었는지 확인
- FRI 프로토콜의 핵심

8.4 FRI 프로토콜 (Fast Reed-Solomon IOP)

8.4.1 FRI 개요

목표: 다항식 f(x)의 차수가 d 이하임을 증명

핵심 아이디어:

```
f(x) → f₁ (x) → f₂ (x) → ... → fₖ(x) = 상수

각 단계에서 차수를 절반으로 줄임
```

8.4.2 FRI 폴딩 (Folding)

단계 i → i+1:

```
fᵢ(x) = fᵢ,even(x²) + x · fᵢ,odd(x²)

도전 값 α 받기

fᵢ₊₁ (x) = fᵢ,even(x) + α · fᵢ,odd(x)
```

차수 감소:

$\deg(f_i) = d$

$\deg(f_{i+1}) = d/2$

예시:

$f(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3$

$f_even(x^2) = a_0 + a_2 x^2$

$f_odd(x^2) = a_1 + a_3 x^2$

$f(x) = f_even(x^2) + x \cdot f_odd(x^2)$

도전 값 α 받기

$f_1(x) = f_even(x) + \alpha \cdot f_odd(x)$

$= a_0 + a_2 x + \alpha(a_1 + a_3 x)$

$= (a_0 + \alpha a_1) + (a_2 + \alpha a_3)x$

$\deg(f_1) = 1$ (원래 3에서 절반으로 감소)

8.4.3 FRI 쿼리

검증 과정:

1. 무작위 위치 x 선택
2. 각 라운드에서 $f_i(x)$, $f_i(-x)$ 요청
3. 폴딩 관계 확인:

$f_{i+1}(x^2) = (f_i(x) + f_i(-x))/2 + \alpha \cdot (f_i(x) - f_i(-x))/(2x)$

보안:

- 쿼리 수 q → 보안 수준 ~2^{-q}
- 40개 쿼리 → ~2^{-40} 보안

8.4.4 zkVM에서의 FRI

```
추적 다항식 p(x)

↓ 제약 조건 적용

제약 다항식 q(x)

↓ FRI 증명

q(x)의 차수가 낮음을 증명

↓

추적이 올바름을 증명
```

8.5 머클 트리 (Merkle Tree)

8.5.1 머클 트리 구조

```
Root (커밋먼트)

/ \

H1 2   H3 4

/ \ / \

H1   H2   H3   H4

| | | |
```

```
d₁   d₂   d₃   d₄
```

구성:

1. 리프: 데이터의 해시
2. 중간 노드: 자식 노드들의 해시
3. 루트: 전체 데이터의 커밋먼트

8.5.2 머클 경로 (Merkle Path)

증명: 특정 데이터가 트리에 포함되었음을 증명

예시 (d_2 증명):

```
경로: [H₁ , H₃₄ ]

검증:

1. H₂  = Hash(d₂ )

2. H₁₂  = Hash(H₁  || H₂ )

3. Root = Hash(H₁₂  || H₃₄ )

4. Root가 커밋먼트와 일치하는지 확인
```

복잡도:

- 경로 길이: O(log n)
- 검증 시간: O(log n)

8.5.3 zkVM에서의 사용

```
추적 데이터 → 머클 트리 → 루트 (커밋먼트)
```

```
↓

쿼리 위치 → 데이터 + 머클 경로

↓

검증자가 경로 검증
```

8.6 Fiat-Shamir 변환

8.6.1 상호작용 프로토콜

원래 프로토콜:

```
1. Prover → Commitment → Verifier

2. Verifier → Challenge → Prover

3. Prover → Response → Verifier

4. Verifier → Accept/Reject
```

문제: 상호작용 필요 (온라인 검증자)

8.6.2 Fiat-Shamir 변환

변환된 프로토콜:

```
1. Prover → Commitment

2. Challenge = Hash(Commitment) // 해시로 도전 값 생성

3. Prover → Response

4. Verifier → Accept/Reject
```

장점:

- 비상호작용 (오프라인 검증 가능)
- 도전 값이 예측 불가능
- 결정론적 (같은 커밋먼트 → 같은 도전 값)

8.6.3 zkVM에서의 구현

```
let mut challenger = Challenger::from_hasher(vec![], hash);

// 커밋먼트 관찰
challenger.observe(commitment);

// 도전 값 샘플링
let challenge = challenger.sample(); // Hash(commitment)
```

보안:

- 해시 함수의 무작위 오라클 가정
- 증명자가 도전 값을 예측할 수 없음

8.7 AIR 제약 조건의 수학

8.7.1 제약 다항식

정의:

```
각 제약 조건 → 다항식 Cᵢ(x)
```

```
올바른 추적 → Cᵢ(x) = 0 (모든 x에서)
```

예시 (Add 제약):

```
제약: next[0] = local[0] + local[1]

다항식: C(x) = p_next,0(x) - p_local,0(x) - p_local,1(x)

올바른 추적 → C(x) = 0
```

8.7.2 몫 다항식 (Quotient Polynomial)

정의:

```
C(x) = Q(x) · Z(x)

여기서 Z(x) = (x - x₀)(x - x₁ )...(x - xₙ₋₁ )
```

의미:

- C(x)가 모든 추적 지점에서 0
- C(x)가 Z(x)로 나누어떨어짐
- Q(x)의 차수가 낮음

증명:

```
1. Q(x) 계산

2. Q(x)의 차수가 낮음을 FRI로 증명

3. 제약 조건이 만족됨을 증명
```

8.7.3 조합 제약 (Composition)

여러 제약 조건 결합:

```
C(x) = α₁ ·C₁ (x) + α₂ ·C₂ (x) + ... + αₘ·Cₘ(x)

여기서 α₁ , α₂ , ..., αₘ은 무작위 도전 값
```

이유:

- 모든 제약을 하나의 다항식으로 결합
- 효율적인 증명 생성
- 보안 유지 (무작위 선형 결합)

8.8 보안 분석

8.8.1 건전성 (Soundness)

정리: 잘못된 추적에 대해 증명을 생성할 확률은 무시할 수 있을 정도로 작다.

증명 스케치:

```
1. 잘못된 추적 → 최소 하나의 제약 조건 위반

2. 제약 다항식 C(x) ≠ 0 (어떤 x에서)

3. FRI가 낮은 차수를 거부할 확률 높음

4. 쿼리가 위반을 발견할 확률 높음
```

보안 수준:

```
ϵ ≈ 2^(-num_queries) × (degree / field_size)
```

8.8.2 완전성 (Completeness)

정리: 올바른 추적에 대해 항상 증명을 생성할 수 있다.

증명:

```
1. 올바른 추적 → 모든 제약 조건 만족
2. 제약 다항식 C(x) = 0 (모든 x에서)
3. 몫 다항식 Q(x) 존재
4. FRI가 성공적으로 완료
```

8.8.3 영지식성 (Zero-Knowledge)

주의: 이 구현은 영지식성을 제공하지 않습니다!

영지식성 추가 방법:

```
1. 추적에 무작위 값 추가
2. 블라인딩 다항식 사용
3. 추가 프로토콜 라운드
```

8.9 복잡도 분석

8.9.1 시간 복잡도

| 작업 | 복잡도 | 설명 |

|------|--------|------|

| 추적 생성 | O(n) | n = 명령어 수 |

| 다항식 보간 | O(n log n) | FFT 사용 |

| 제약 평가 | O(n) | 각 행에서 평가 |

| FRI 증명 | O(n log n) | 로그 라운드 |

| 증명 검증 | O(q log n) | q = 쿼리 수 |

8.9.2 공간 복잡도

| 데이터 | 크기 | 설명 |

|--------|------|------|

| 추적 | O(n) | n = 행 수 |

| 다항식 | O(n) | 계수 저장 |

| 머클 트리 | O(n) | 리프 + 중간 노드 |

| 증명 | O(log n) | 커밋먼트 + 경로 |

8.9.3 통신 복잡도

증명 크기:

```
Size = O(log n) × (hash_size + field_size × num_queries)
```

예시:

```
n = 1024 (추적 크기)

log n = 10 (FRI 라운드)

hash_size = 32 bytes

field_size = 4 bytes

num_queries = 40
```

```
Size ≈ 10 × (32 + 4 × 40) = 10 × 192 = 1920 bytes ≈ 2KB
```

8.10 수학적 최적화

8.10.1 Blowup Factor

정의:

```
blowup = evaluation_domain_size / trace_size
```

트레이드오프:

- 큰 blowup: 더 안전, 더 느림
- 작은 blowup: 덜 안전, 더 빠름

이 구현:

```
log_blowup = 1 → blowup = 2
```

8.10.2 쿼리 수 최적화

보안 vs 효율성:

```
더 많은 쿼리: 더 안전, 더 큰 증명

더 적은 쿼리: 덜 안전, 더 작은 증명
```

이 구현:

```
num_queries = 40 → ~80비트 보안
```

8.10.3 필드 선택

Mersenne31의 장점:

- 빠른 모듈로 연산
- 32비트 정수로 표현
- 충분한 크기 (2^{31}-1)

다른 선택:

- Goldilocks (2^{64} - 2^{32} + 1): 더 큰 필드
- BabyBear (2^{31} - 2^{27} + 1): 비슷한 크기

8.11 요약

zkVM의 수학적 기초는 다음을 포함합니다:

1. **유한체**: 오버플로우 없는 안전한 연산
2. **다항식**: 추적을 수학적 객체로 표현
3. **Reed-Solomon 코드**: 낮은 차수 테스트
4. **FRI**: 효율적인 차수 증명
5. **머클 트리**: 간결한 커밋먼트
6. **Fiat-Shamir**: 비상호작용 변환

이러한 수학적 도구들이 결합되어 효율적이고 안전한 zkVM을 만듭니다.